

Aula 2 - Multiplayer e leaderboard

Guia do Aluno - versao detalhada

O que voce vai construir

Voce vai jogar junto com a turma na rede. Vai ver as cobras dos colegas (cada uma com a cor e o emoji do dono) e um ranking que atualiza ao vivo. O professor roda o servidor; voce acessa pelo navegador em `http://IP-DO-PROFESSOR:5000`.

Hoje voce implementa 4 partes: (1) a conexao do navegador com o servidor, (2) salvar o score no banco, (3) consultar o ranking e (4) desenhar as cobras dos outros jogadores.

Antes de comecar: como o multiplayer funciona

Cliente, servidor e as duas formas de conversar

Seu navegador e o CLIENTE. A maquina do professor roda o SERVIDOR (um programa em C#). Eles conversam de duas formas:

- HTTP (fetch): uma pergunta e uma resposta, e fecha. Usamos para salvar score (POST) e pedir o ranking (GET).
- WebSocket: uma linha que fica ABERTA nos dois sentidos, como uma ligacao. Usamos para o jogo em tempo real (posicao das cobras 10x por segundo).

JSON: o 'idioma' das mensagens

As mensagens viajam como texto no formato JSON (parece um objeto JavaScript).

`JSON.stringify(obj)` transforma um objeto em texto (para enviar).

`JSON.parse(texto)` transforma o texto de volta em objeto (ao receber).

Ferramentas que ja existem e voce vai usar

- `ws` - a variavel que vai guardar o WebSocket.
- `handlers` - um objeto que liga um 'tipo' de mensagem a uma funcao (ex: `handlers['gameState']`).
- `enviarParaServidor(tipo, dados)` - manda uma mensagem pelo WebSocket.
- No servidor (C#): `_db` (o banco), `GameSession` (uma partida), `ObterTop10()` (consulta pronta), `_gameLoop.BroadcastLeaderboard(top)` (avisa todos do novo ranking).

Os 4 desafios (passo a passo)

Desafio 1: conectar() - abrir o WebSocket (arquivo `wwwroot/network.js`)

Entendendo o problema

Quando a partida comeca, o jogo chama `conectar(nome, perfil)`. Sua missao: abrir a linha com o servidor, se apresentar e ficar ouvindo as mensagens dele.

Um WebSocket tem 4 momentos: criar a conexao, `'onopen'` (abriu), `'onmessage'` (chegou mensagem) e `'onclose'` (caiu).

O que voce precisa saber antes

O endereço do WebSocket

Use 'ws://' + location.host + '/ws'. O location.host é o mesmo endereço que serviu a página (o IP do professor + porta). Assim funciona em qualquer máquina.

Repare: é 'ws://' (WebSocket), não 'http://'.

O 'perfil' (cor, emoji, tag) já vem pronto de quem chamou a função. Você só o repassa dentro da mensagem 'entrar'.

Construindo passo a passo**Passo 1 — Crie a conexão**

Guarde o WebSocket na variável global ws.

```
ws = new WebSocket("ws://" + location.host + "/ws");
```

Passo 2 — Ao abrir (onopen), apresente-se

Assim que a linha abrir, mande uma mensagem do tipo 'entrar' com seu nome e perfil. Lembre de transformar o objeto em texto com JSON.stringify.

```
ws.onopen = function () {
  ws.send(JSON.stringify({ tipo: "entrar", nome: nome, perfil: perfil }));
};
```

Passo 3 — Ao receber (onmessage), entregue ao handler certo

Cada mensagem tem um campo 'tipo'. Transforme o texto em objeto e, se existir um handler para aquele tipo, chame-o passando a mensagem.

```
ws.onmessage = function (evento) {
  var msg = JSON.parse(evento.data);
  if (handlers[msg.tipo]) {
    handlers[msg.tipo](msg);
  }
};
```

Passo 4 — Ao cair (onclose), avise no console

So para você saber que a conexão terminou.

```
ws.onclose = function () {
  console.log("[rede] conexão encerrada.");
};
```

Confira: a função completa

```
function conectar(nome, perfil) {
  ws = new WebSocket("ws://" + location.host + "/ws");

  ws.onopen = function () {
    ws.send(JSON.stringify({ tipo: "entrar", nome: nome, perfil: perfil }));
  };

  ws.onmessage = function (evento) {
    var msg = JSON.parse(evento.data);
    if (handlers[msg.tipo]) handlers[msg.tipo](msg);
  };

  ws.onclose = function () {
    console.log("[rede] conexão encerrada.");
  };
}
```

Como saber que funcionou

No Console (F12) aparece a conexao; na tela do professor surge a sua entrada. As cobras dos outros aparecem depois do Desafio 4.

Se der erro

Mensagem/sintoma: "InvalidOperationException: still in CONNECTING state" ou a conexao falha.

Causa: voce tentou enviar 'entrar' antes do onopen (a linha ainda nao atendeu).

Solucao: coloque o ws.send(...) DENTRO de ws.onopen.

Se der erro

Mensagem/sintoma: nada acontece e nao ha erro.

Causa: voce escreveu 'http:/' em vez de 'ws:/' no endereco.

Solucao: use 'ws:/' + location.host + '/ws'.

Desafio 2: POST /api/score - salvar a partida no banco (ScoreController.cs)**Entendendo o problema**

Quando a cobra morre, o cliente manda o score para o servidor. Sua missao, no metodo PostScore, e gravar essa partida no banco de dados e avisar todos do novo ranking.

O servidor recebe um 'dto' (um pacote com Nome, Score, Emoji, Tag, Cor). Voce transforma isso numa linha da tabela GameSessions.

O que voce precisa saber antes**EF Core: salvar e como 'Add' + 'SaveChanges'**

_db.GameSessions.Add(x) coloca x na fila para salvar.

await _db.SaveChangesAsync() grava de verdade no arquivo game.db.

O 'await' espera a gravacao terminar antes de seguir.

Depois de salvar, chame ObterTop10() (consulta pronta) e passe o resultado para _gameLoop.BroadcastLeaderboard(top): isso manda o ranking novo para a tela de todos.

Construindo passo a passo**Passo 1 — Crie a linha do banco a partir do dto**

Copie cada campo do dto para um novo GameSession.

```
var sessao = new GameSession {
    NomeJogador = dto.Nome,
    Score = dto.Score,
    Emoji = dto.Emoji,
    Tag = dto.Tag,
    Cor = dto.Cor
};
```

Passo 2 — Salve no banco

Add coloca na fila; SaveChangesAsync grava. Nao esqueca o await.

```
_db.GameSessions.Add(sessao);
await _db.SaveChangesAsync();
```

Passo 3 — Atualize o ranking ao vivo

Pegue o top-10 e mande para todos.

```
var top = await ObterTop10();
_gameLoop.BroadcastLeaderboard(top);
```

Confira: a funcao completa

```
// dentro de PostScore, depois da validacao do dto:
var sessao = new GameSession {
    NomeJogador = dto.Nome,
    Score = dto.Score,
    Emoji = dto.Emoji,
    Tag = dto.Tag,
    Cor = dto.Cor
};
_db.GameSessions.Add(sessao);
await _db.SaveChangesAsync();

var top = await ObterTop10();
_gameLoop.BroadcastLeaderboard(top);
return Ok(new { ok = true });
```

Como saber que funcionou

O arquivo game.db cresce; em /swagger, o GET /api/leaderboard mostra a partida salva; o painel Ranking atualiza para todos.

Se der erro

Mensagem/sintoma: "SQLite Error 1: 'no such table: GameSessions'".

Causa: o banco foi criado antes de a tabela existir.

Solucao: pare o servidor, apague o arquivo game.db e rode 'dotnet run' de novo.

Se der erro

Mensagem/sintoma: o score nao salva e nada acontece.

Causa: voce esqueceu o 'await' no SaveChangesAsync.

Solucao: escreva 'await _db.SaveChangesAsync;'.

Desafio 3: GET /api/leaderboard - devolver o top-10 (ScoreController.cs)

Entendendo o problema

Para mostrar o ranking, o servidor precisa entregar os 10 maiores scores. Voce faz isso no metodo GetLeaderboard.

O que voce precisa saber antes

LINQ: perguntar ao banco quase em ingles

OrderByDescending(s => s.Score) ordena do MAIOR para o menor.

Take(10) pega so os 10 primeiros.

O metodo auxiliar ObterTop10() ja monta essa consulta para voce.

Construindo passo a passo

Passo 1 — Pegue o top-10 e devolva

Use o auxiliar pronto e responda com Ok(...).

```
var top = await ObterTop10();
return Ok(top);
```

Confira: a funcao completa

```
public async Task<IActionResult> GetLeaderboard() {
    var top = await ObterTop10();
    return Ok(top);
}
```

Como saber que funcionou

O painel 'Ranking' passa a mostrar os scores, do maior para o menor.

Se der erro

Mensagem/sintoma: o ranking aparece de cabeca para baixo (piores no topo).

Causa: a consulta usou OrderBy (crescente) em vez de OrderByDescending.

Solucao: no auxiliar ObterTop10, use OrderByDescending(s => s.Score).

Desafio 4: desenharJogadoresRemotos() - ver os outros jogadores (game.js)**Entendendo o problema**

O servidor manda, 10 vezes por segundo, a lista de todos os jogadores com a posicao e o perfil de cada um. Sua missao: desenhar cada cobra (menos a sua) com a cor do dono, e escrever o emoji e o nome.

Voce desenha no 'ctx', que e o pincel do canvas. As posicoes vem em celulas; multiplique por TAMANHO para virar pixels.

O que voce precisa saber antes**O formato de cada jogador**

Cada item da lista tem: jog.nome, jog.segmentos (lista de {x,y}, sendo o [0] a cabeca) e jog.perfil (com .cor, .emoji e .tag).

PULE a sua propria cobra (voce ja a desenha localmente): if (jog.nome === nomeJogador) continue;

Desenhar no canvas

ctx.fillStyle = cor - escolhe a cor do pincel.

ctx.fillRect(px, py, largura, altura) - pinta um retangulo.

ctx.fillText(texto, px, py) - escreve um texto (use ctx.font para o tamanho).

Construindo passo a passo**Passo 1 — Guarde a lista e percorra os jogadores**

A funcao recebe a lista; guarde-a e percorra cada jogador, pulando voce mesmo.

```
if (jogadores) jogadoresRemotos = jogadores;
ctx.textAlign = "center";
for (var j = 0; j < jogadoresRemotos.length; j++) {
    var jog = jogadoresRemotos[j];
    if (jog.nome === nomeJogador) continue;
    var segs = jog.segmentos || [];
    if (segs.length === 0) continue;
```

Passo 2 — Pinte o corpo com a cor do dono

Para cada segmento, pinte um quadradinho na cor do perfil.

```

ctx.fillStyle = jog.perfil.cor;
for (var i = 0; i < segs.length; i++) {
  ctx.fillRect(segs[i].x * TAMANHO + 1, segs[i].y * TAMANHO + 1,
    TAMANHO - 2, TAMANHO - 2);
}

```

Passo 3 — Escreva o emoji acima da cabeça e o nome embaixo

A cabeça é `segs[0]`. Calcule o centro em pixels e desenhe o emoji e o nome.

```

var cabeca = segs[0];
var px = cabeca.x * TAMANHO + TAMANHO / 2;
var py = cabeca.y * TAMANHO;
ctx.font = TAMANHO + "px sans-serif";
ctx.fillText(jog.perfil.emoji, px, py - 2);
ctx.font = "11px sans-serif";
ctx.fillStyle = "#ffffff";
var rotulo = jog.nome + (jog.perfil.tag ? " [" + jog.perfil.tag + "]" : "");
ctx.fillText(rotulo, px, py + TAMANHO + 12);
}
ctx.textAlign = "start";

```

Confira: a funcao completa

```

function desenharJogadoresRemotos(jogadores) {
  if (jogadores) jogadoresRemotos = jogadores;
  ctx.textAlign = "center";
  for (var j = 0; j < jogadoresRemotos.length; j++) {
    var jog = jogadoresRemotos[j];
    if (jog.nome === nomeJogador) continue;
    var segs = jog.segmentos || [];
    if (segs.length === 0) continue;
    ctx.fillStyle = jog.perfil.cor;
    for (var i = 0; i < segs.length; i++) {
      ctx.fillRect(segs[i].x * TAMANHO + 1, segs[i].y * TAMANHO + 1,
        TAMANHO - 2, TAMANHO - 2);
    }
    var cabeca = segs[0];
    var px = cabeca.x * TAMANHO + TAMANHO / 2;
    var py = cabeca.y * TAMANHO;
    ctx.font = TAMANHO + "px sans-serif";
    ctx.fillText(jog.perfil.emoji, px, py - 2);
    ctx.font = "11px sans-serif";
    ctx.fillStyle = "#ffffff";
    var rotulo = jog.nome + (jog.perfil.tag ? " [" + jog.perfil.tag + "]" : "");
    ctx.fillText(rotulo, px, py + TAMANHO + 12);
  }
  ctx.textAlign = "start";
}

```

Como saber que funcionou

As cobras dos colegas aparecem na tela, cada uma com a sua cor e o seu emoji.

Se der erro

Mensagem/sintoma: "Cannot read properties of undefined (reading 'cor')".

Causa: voce leu `jog.perfil.cor` sem garantir que o item tem perfil e segmentos.

Solucao: use a lista `'jogadores'` que vem do servidor; cada item tem `.perfil` e `.segmentos`.

Se der erro

Mensagem/sintoma: a sua cobra aparece duplicada/estranha.

Causa: faltou pular o proprio jogador.

Solucao: adicione 'if (jog.nome === nomeJogador) continue;'.
'

Personalize sua identidade no multiplayer

Entrem todos ao mesmo tempo e olhem a tela: a cobra do colega tem a cor e o emoji que ELE escolheu. Ajuste a SUA identidade no config.js:

- emoji_cobra - o emoji que flutua acima da sua cabeca para os outros.
- cor_multiplayer - deixe "" para o servidor gerar pelo seu nome, ou escolha um #hex.
- tag_jogador - uma tag curta: "speed runner", "sobrevivente".

Como saber que funcionou

Sua cobra passa a exibir o novo emoji, a nova cor e a tag no leaderboard de todos. Isso vai aparecer tambem no torneio da Aula 4.

Glossario

Cliente / Servidor: cliente = seu navegador; servidor = o programa do professor que atende todos.

WebSocket: uma conexao aberta nos dois sentidos, como uma ligacao telefonica continua.

HTTP / fetch: uma pergunta e uma resposta, e encerra (POST envia, GET consulta).

JSON: formato de texto para enviar objetos. stringify = objeto->texto; parse = texto->objeto.

handler: uma funcao registrada para tratar um tipo de mensagem que chega.

EF Core: o tradutor entre o C# e o banco; voce fala em objetos, ele fala SQL.

Add / SaveChanges: Add poe na fila para salvar; SaveChanges grava de verdade no banco.

LINQ: o jeito do C# de consultar dados (OrderByDescending, Take, Where).

await: espera uma operacao demorada (gravar/consultar) terminar antes de seguir.

ctx (contexto do canvas): o 'pincel' com que se desenha no canvas (fillRect, fillText).